

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут» імені Ігоря Сікорського
Факультет інформатики та обчислювальної техніки
Кафедра автоматики та управління в технічних системах

Лабораторна робота № 9

із дисципліни: «Технології розроблення програмного забезпечення»

тема: Взаємодія компонентів системи

Виконав:

студент групи ІА-34

Єресько І.С

Перевірив:

Мягкий Михайло Юрійович

Тема: Взаємодія компонентів системи.

Мета: Вивчити види взаємодії додатків (Client-Server, Peer-to-Peer, Service oriented Architecture), та реалізувати в проєктованій системі одну із архітектур.

Короткі теоретичні відомості:

Клієнт-серверна архітектура — це модель побудови розподілених систем, де функціональність розділена між двома типами вузлів: клієнтами, які ініціюють запити на отримання даних або послуг, та серверами, які ці запити обробляють, зберігають дані та надсилають відповіді. Ця модель забезпечує централізоване керування ресурсами та безпекою на стороні сервера, дозволяючи при цьому мати безліч клієнтів з різними інтерфейсами (веб, мобільні, десктопні), що спрощує масштабування та адміністрування системи.

Архітектура Peer-to-Peer (однорангова) архітектура — це децентралізована мережева модель, де всі учасники (вузли або "піри") мають рівні права і можуть виступати одночасно як клієнтами, так і серверами, обмінюючись ресурсами без посередництва центрального сервера. Така структура забезпечує високу відмовостійкість та масштабованість, оскільки вихід з ладу одного вузла не зупиняє роботу всієї мережі, а додавання нових учасників лише збільшує загальну потужність та доступність ресурсів системи.

Сервіс-орієнтована архітектура (SOA) — це підхід до проєктування програмного забезпечення, де система будується як набір слабко пов'язаних, автономних сервісів, які взаємодіють між собою через чітко визначені мережеві протоколи та інтерфейси (часто через корпоративну шину сервісів — ESB). Головною метою SOA є забезпечення повторного використання бізнес-логіки та інтеграція різнорідних систем, що дозволяє великим підприємствам гнучко компонувати складні бізнес-процеси з існуючих сервісів незалежно від платформи їх реалізації.

Мікросервісна архітектура — це сучасна еволюція SOA, де додаток розробляється як сукупність невеликих, незалежних сервісів, кожен з яких відповідає за одну конкретну бізнес-функцію, має власну базу даних та розгортається окремо. Такий підхід дозволяє командам розробників працювати автономно, використовувати різні технологічні стеки для різних задач, а також масштабувати лише ті частини системи, які цього потребують, що значно підвищує гнучкість розробки та стійкість додатка до збоїв.

Обрана тема лабораторної роботи: **SQL IDE**

8. Powershell terminal (strategy, command, abstract factory, bridge, interpreter, client-server)

Термінал для powershell повинен нагадувати типовий термінал з можливістю налаштування кольорів синтаксичних конструкцій, розміру вікна, фону вікна, а також виконання команд powershell і виконуваних файлів, а також працювати в декількох вікнах терміналу (у вкладках або одночасно шляхом розділення вікна).

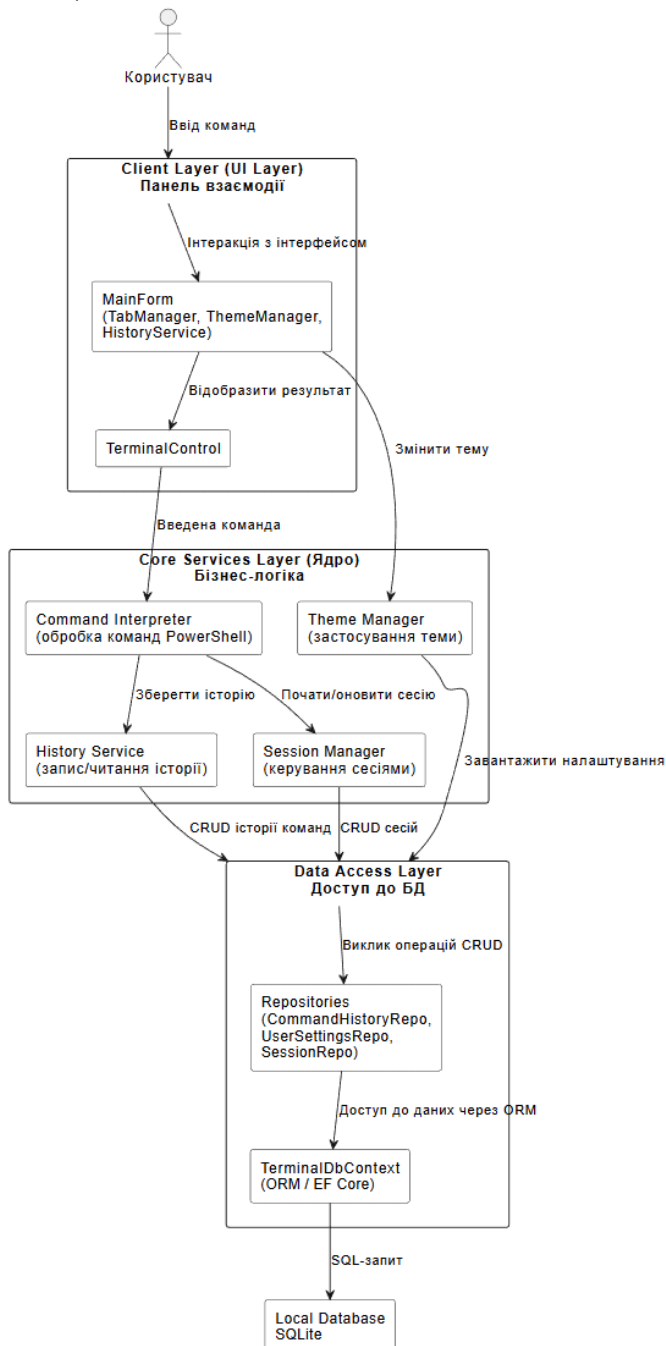


Рис 1:client-server

Ця діаграма демонструє детальну клієнт-серверну архітектуру застосунку PowerShell Terminal, у якій чітко розділені всі рівні системи та напрямки взаємодії між ними. Взаємодія починається з користувача, який вводить команди у клієнтський інтерфейс. Client Layer включає MainForm — центральне вікно застосунку, що керує вкладками, темами та історією, а також TerminalControl, який виступає безпосереднім полем введення та виведення тексту. На цьому рівні відбувається лише обробка введення користувача та відображення результатів, тоді як сама логіка виконання команд передається на серверну частину — Core Services Layer.

У шарі Core Services зосереджена вся бізнес-логіка застосунку. Command Interpreter приймає введені команди та визначає їхню структуру, виконує інтерпретацію та керує процесом обробки. History Service відповідає за фіксацію кожної виконаної команди, збереження її результатів, часу запуску та можливих помилок. Theme Manager у цьому шарі керує зовнішнім виглядом застосунку, дозволяючи змінювати кольорові схеми, шрифти та інші елементи інтерфейсу. Session Manager керує життєвим циклом сесій, фіксує їх початок та завершення, а також дозволяє застосунку працювати з кількома активними сеансами.

Усі сервіси ядра взаємодіють із Data Access Layer, який забезпечує структурований доступ до бази даних. У цьому шарі зосереджені репозиторії, кожен з яких відповідає за власний тип даних: історію команд, налаштування користувача та інформацію про сесії. Репозиторії передають запити до TerminalDbContext — об'єкта, що виступає проміжною ланкою між бізнес-логікою та фізичною базою даних. TerminalDbContext формує SQL-операції, керує транзакціями та забезпечує зв'язок із локальною базою SQLite.

Локальна база SQLite є серверною частиною архітектури, де зберігаються всі постійні дані системи: історія виконання команд, параметри оформлення, інформація про активні та завершені сесії. Через DbContext до неї надходять CRUD-запити, згенеровані репозиторіями. Завдяки цьому система забезпечує централізовану обробку та зберігання даних, а також відокремлює клієнтську частину від внутрішньої серверної логіки.

Уся діаграма в цілому відображає повний шлях руху даних — від моменту введення команд у клієнті до їхньої обробки в ядрових сервісах та остаточного збереження в базі даних. Такий підхід забезпечує масштабованість, модульність і чіткий розподіл відповідальності між рівнями системи, що є ключовими характеристиками класичної клієнт-серверної архітектури.

Висновок

У процесі вивчення взаємодії компонентів систем було проаналізовано декілька основних архітектурних підходів: клієнт-серверну модель, однорангову мережу та сервісно-орієнтовану архітектуру. Кожна з них має свої сильні та слабкі сторони, тому вибір залежить від конкретних вимог до масштабованості, безпеки, централізації управління та стійкості до відмов. У проєктованій системі було реалізовано клієнт-серверну архітектуру, яка забезпечила чіткий розподіл обов'язків між клієнтською та серверною частинами, централізоване зберігання даних, зручне масштабування та порівняно просте впровадження механізмів аутентифікації й авторизації. Отриманий практичний досвід підтвердив доцільність використання саме цієї моделі для більшості класичних веб-додатків і корпоративних систем, водночас показавши, у яких випадках доцільно розглядати альтернативні підходи. Таким чином, вивчення та практична реалізація різних типів взаємодії компонентів значно поглибили розуміння сучасних архітектурних рішень і критеріїв їх вибору.

Контрольні питання

1. Що таке клієнт-серверна архітектура? Клієнт-серверна архітектура — це модель побудови розподілених систем, де функціональне навантаження розділене між постачальниками послуг (серверами) і замовниками послуг (клієнтами). Клієнти ініціюють запити на отримання даних або виконання операцій, а сервери обробляють ці запити, виконують обчислення, звертаються до баз даних і надсилають результат назад, що дозволяє централізувати зберігання даних і логіку обробки, забезпечуючи при цьому доступ до системи з багатьох робочих місць.
2. Розкажіть про сервіс-орієнтовану архітектуру (SOA). Сервіс-орієнтована архітектура (SOA) — це підхід до розробки програмного забезпечення, в якому додатки будуються з набору дискретних, слабо пов'язаних сервісів, що взаємодіють між собою через чітко визначені інтерфейси та протоколи, часто через мережу. Основна ідея SOA полягає в повторному використанні бізнес функцій, інкапсульованих у сервісах, що дозволяє швидко створювати нові додатки шляхом комбінування існуючих компонентів, незалежно від платформи чи мови програмування, на якій вони реалізовані.
3. Якими принципами керується SOA? SOA керується набором принципів, ключовими з яких є: слабка зв'язність (сервіси мають мінімальні залежності

один від одного), абстракція (внутрішня логіка прихована, доступний лише інтерфейс), повторне використання (сервіси проєктуються для використання у багатьох бізнес-процесах), автономність (сервіси контролюють власне середовище виконання) та сумісність (здатність взаємодіяти незалежно від технологічної платформи через стандартні протоколи).

4. Як між собою взаємодіють сервіси в SOA? У SOA сервіси взаємодіють за допомогою повідомлень, що передаються через стандартизовані протоколи (найчастіше SOAP або REST over HTTP), і часто використовують проміжний шар — корпоративну сервісну шину (Enterprise Service Bus, ESB). ESB виступає як комунікаційний хаб, який маршрутизує повідомлення, трансформує формати даних (наприклад, з XML у JSON) і забезпечує оркестрацію викликів, дозволяючи сервісам спілкуватися, навіть якщо вони використовують різні транспортні протоколи.

5. Як розробники взнають про існуючі сервіси і як робити до них запити? Для виявлення сервісів у SOA використовується реєстр сервісів (Service Registry) або каталог, де публікуються описи доступних сервісів, їхні інтерфейси (контракти, наприклад, WSDL для SOAP або Swagger/OpenAPI для REST) та політики використання. Розробники знаходять потрібний сервіс у реєстрі, вивчають його контракт, який описує доступні методи та формат даних, і на основі цього формують клієнтський код для виконання запитів.

6. У чому полягають переваги та недоліки клієнт-серверної моделі? Перевагами є централізоване керування даними та безпекою, легкість обслуговування (оновлення сервера автоматично доступне всім клієнтам) і масштабованість серверної частини. Недоліками є наявність єдиної точки відмови (якщо сервер падає, робота зупиняється), можливі перевантаження мережі при великій кількості запитів та висока вартість серверного обладнання і його адміністрування.

7. У чому полягають переваги та недоліки однорангової моделі взаємодії (P2P)? Перевагами P2P є висока відмовостійкість (немає центрального сервера, вихід з ладу одного вузла не критичний), легка масштабованість (кожен новий клієнт додає ресурси в мережу) та відсутність потреби у дорогому серверному обладнанні. Недоліками є складність адміністрування та забезпечення безпеки, неможливість гарантувати доступність конкретного файлу чи ресурсу в будь-який момент часу та ускладнена логіка пошуку даних у мережі.

8. Що таке мікросервісна архітектура? Мікросервісна архітектура — це стиль проєктування, де єдиний додаток розробляється як набір невеликих сервісів, кожен з яких працює у власному процесі, відповідає за окрему бізнес-функцію (Bounded Context) і може бути розгорнутий незалежно. Це дозволяє використовувати різні технології для різних сервісів, спрощує масштабування (можна масштабувати лише "гарячі" сервіси) та прискорює розробку, оскільки

невеликі команди можуть працювати паралельно над різними мікросервісами.

9. Які протоколи використовуються для обміну даними в мікросервісній архітектурі? Для синхронної взаємодії найчастіше використовують легковагові протоколи HTTP/HTTPS (стиль REST) або gRPC (для високопродуктивного міжсервісного зв'язку). Для асинхронної взаємодії широко застосовуються протоколи черг повідомлень, такі як AMQP (RabbitMQ) або Kafka, що дозволяє зменшити зв'язність сервісів і підвищити стійкість системи до пікових навантажень.

10. Чи можна назвати підхід сервіс-орієнтованою архітектурою, коли ми в проєкті між шаром веб-контролерів та шаром доступу до даних реалізуємо шар бізнес-логіки у вигляді сервісів? Строго кажучи, ні, це називається Service Layer Pattern (патерн Шару Сервісів) у рамках монолітної або шаруватої архітектури, а не повноцінною SOA. Хоча це використовує принципи розділення відповідальності, характерні для SOA, справжня Сервіс-Орієнтована Архітектура передбачає, що сервіси є розподіленими, автономними компонентами, які можуть бути розгорнуті на різних серверах і використовуватися різними, незалежними додатками, а не просто класами всередині однієї програми.